NimbusHR Engineering Guidelines

Version 3.1 | Last Updated: January 2025

Engineering Principles

Core Philosophy

NimbusHR values **simplicity, maintainability, and testability** in all software systems. We believe that well-crafted code is:

- **Simple**: Easy to understand and reason about
- **Maintainable**: Can be modified and extended without breaking existing functionality
- **Testable**: Designed with automated testing in mind from the start
- **Scalable**: Capable of handling growth in users, data, and features
- **Secure**: Built with security best practices at every layer

Code Quality Standards

Write for Readability:

- Use descriptive variable and function names (`calculateMonthlyRecurringRevenue` not `calcMRR`)
- Keep functions small and focused (ideally under 50 lines)
- Write self-documenting code; add comments only when necessary
- Follow language-specific style guides (PEP 8 for Python, Airbnb for JavaScript)

Embrace Simplicity (KISS Principle):

- Choose the simplest solution that solves the problem
- Avoid over-engineering and premature optimization
- Don't add abstractions until you need them (YAGNI)
- Prefer proven technology over experimental frameworks

Design for Testability:

- Aim for 80%+ test coverage on critical paths
- Write unit tests for individual functions
- Include integration tests for component interactions
- Use dependency injection to make components testable
- Test edge cases and error conditions, not just happy paths

Technical Stack

- **Backend**: Python 3.11+ (FastAPI), Node.js 20+ for real-time features
- **Frontend**: React 18+ with TypeScript, Tailwind CSS
- **Database**: PostgreSQL 15+, Redis 7+ (cache)
- **Infrastructure**: AWS (EKS/Kubernetes), GitHub Actions, ArgoCD
- **Monitoring**: Datadog, Sentry, OpenTelemetry

Adding new technology requires an RFC (Request for Comments) document and team consensus.

Security Requirements

All engineers are responsible for writing secure code:

- Use OAuth 2.0/OIDC for authentication
- Implement role-based access control (RBAC) on all endpoints
- Encrypt sensitive data at rest (AES-256) and in transit (TLS 1.3)
- Hash passwords with bcrypt (minimum 12 rounds)
- Sanitize all user input to prevent injection attacks
- Never log sensitive information (passwords, tokens, PII)
- Rate limit public endpoints (100 req/min per IP)

Security review required for all authentication/authorization changes.

Performance Targets

- API response time: p95 < 200ms, p99 < 500ms
- Page load time: < 2 seconds on 3G connection
- Database query time: < 100ms for 95% of queries
- Design stateless services for horizontal scaling
- Use caching aggressively (Redis)
- Queue non-urgent work (emails, analytics)

Documentation Standards

Required Documentation:

- README.md in every repository with setup instructions
- API documentation via OpenAPI/Swagger specs
- Architecture Decision Records (ADRs) for significant decisions
- Inline comments for complex business logic
- Migration guides for breaking changes

Code Reviews

Review Requirements

All code changes must be **reviewed by at least one other engineer** before being merged into the main branch. No exceptions, even for urgent hotfixes.

Reviewer Assignment:

- **Small PRs** (<200 lines): 1 reviewer
- **Medium PRs** (200-500 lines): 2 reviewers
- **Large PRs** (500+ lines): 2+ reviewers (consider breaking up)
- **Security-sensitive changes**: Security champion must review

Pull Request Process

1. Create Pull Request:

- Use descriptive title: `[Component] Brief description`
- Complete PR template (What, Why, How, Testing, Screenshots)
- Link related Jira ticket(s)
- Ensure all CI checks pass before requesting review

2. Automated Checks (must pass):

- [ ] Unit tests (90%+ coverage)
- [ ] Integration tests
- [ ] Linting (Pylint, ESLint)
- [ ] Type checking (mypy, TypeScript)
- [ ] Security scanning (Snyk)

3. Review Timeline:

- Reviewers respond within **4 business hours**
- Complete review within **24 hours** (normal priority)
- Urgent PRs: **2 hours** (Slack reviewer directly)

4. Merge:

- All reviewers must approve
- All comments resolved
- All CI checks green
- Use "Squash and Merge" to keep history clean

What to Review

- **Correctness**: Does it work? Edge cases handled? Error handling appropriate?
- **Design**: Fits architecture? Overly complex? Better approaches available?
- **Readability**: Clear names? Logical structure? Would new team member understand?
- **Testing**: Adequate tests? Edge cases covered? Tests actually test the right things?
- **Performance**: Obvious issues? N+1 queries? Caching appropriate?
- **Security**: Input validation? Authentication/authorization? SQL injection or XSS risks?

Feedback Guidelines

Giving Feedback:

- Be kind and constructive; assume positive intent
- Be specific with examples
- Distinguish severity:
  - □ **BLOCKING**: Bugs, security issues (must fix)
  - □ **SUGGESTION**: Improvements (nice to have)
  - 🔍 **NIT**: Minor style preferences (author's choice)
- Explain the "why" behind feedback
- Praise good work explicitly

Example:

```

✗ "This is confusing"

✓ "This function name doesn't indicate it has side effects. Consider

  renaming to `create_and_send_invoice` to make it explicit."

```

Receiving Feedback:

- Don't take it personally
- Respond to every comment
- Ask clarifying questions if confused
- Push back respectfully if you disagree (with reasoning)
- Thank reviewers for catching issues

Deployment Process

CI/CD Pipeline

NimbusHR uses **continuous integration and continuous deployment (CI/CD)**. **Production deployments occur automatically after all checks pass.**

Pipeline Stages:

- **Build**: Compile code, install dependencies
- **Test**: Run unit, integration, and E2E tests
- **Security Scan**: Check for vulnerabilities
- **Deploy to Staging**: Automatic deployment
- **Smoke Tests**: Critical path validation
- **Deploy to Production**: Automatic if all checks pass
- **Post-Deploy Verification**: Health checks and monitoring

Timeline: Merge to `main` → Production in **15-20 minutes**

Deployment Strategies

Rolling Deployment (default for backend):

- 25% of pods updated at a time
- Health checks between batches
- Automatic rollback if health checks fail

Blue-Green Deployment (database schema changes):

- New version deployed alongside old
- Traffic switched after validation
- Old version kept for quick rollback

Canary Deployment (high-risk changes):

- 5% traffic to new version initially
- Gradually increase to 100% over 2 hours
- Monitor error rates and latency

Environments

Development (local): Personal development on engineer's machine

Staging (staging.nimbushr.io):

- Mirrors production environment
- Automated deployments on merge to `main`
- Used for QA and final testing
- Refreshed with production data weekly (anonymized)

Production (app.nimbushr.io):

- Live customer environment
- Automatic deployments after staging validation
- 99.9% uptime SLA
- Multi-region (us-east-1 primary, us-west-2 DR)

Deployment Controls

Feature Flags: Use LaunchDarkly for gradual rollouts

- Enable features for internal users first
- Roll out to 10% → 50% → 100% of customers
- Kill switch for immediate feature disable

Deployment Windows:

- **Preferred**: Tuesday-Thursday, 10 AM - 4 PM EST
- **Avoid**: Fridays, weekends, holidays, end-of-quarter
- **Freeze**: Last week of quarter (exception: critical fixes)

Emergency Deployments: Require approval from on-call engineer and engineering manager

Database Migrations

Migration Process:

1. Write backward-compatible migration (additive changes only)
2. Deploy application code that works with old and new schema
3. Run migration during maintenance window (Sunday 2-4 AM UTC)
4. Deploy code that requires new schema
5. Remove old schema after 1 week

Destructive changes (dropping columns/tables):

- Requires 2-week deprecation period
- Must notify all teams
- Verify no dependencies via code search

Rollback Procedures

Automatic Rollback: Triggered if:

- Health checks fail (>5% error rate)
- Response time increases >50%
- Deployment doesn't complete in 30 minutes

Manual Rollback:

1. Notify #engineering Slack channel
2. Revert via ArgoCD UI or `git revert` + push
3. File incident report
4. Schedule post-mortem

Rollback Time: Target < 10 minutes for critical issues

Monitoring & Alerts

Deployment Dashboards:

- Real-time deployment status in ArgoCD
- Error rates and latency in Datadog
- Application logs in CloudWatch

Alerts (sent to #alerts Slack):

- Deployment failures
- Error rate > 1%
- p95 latency > 500ms
- Database connection pool exhaustion
- Disk usage > 80%